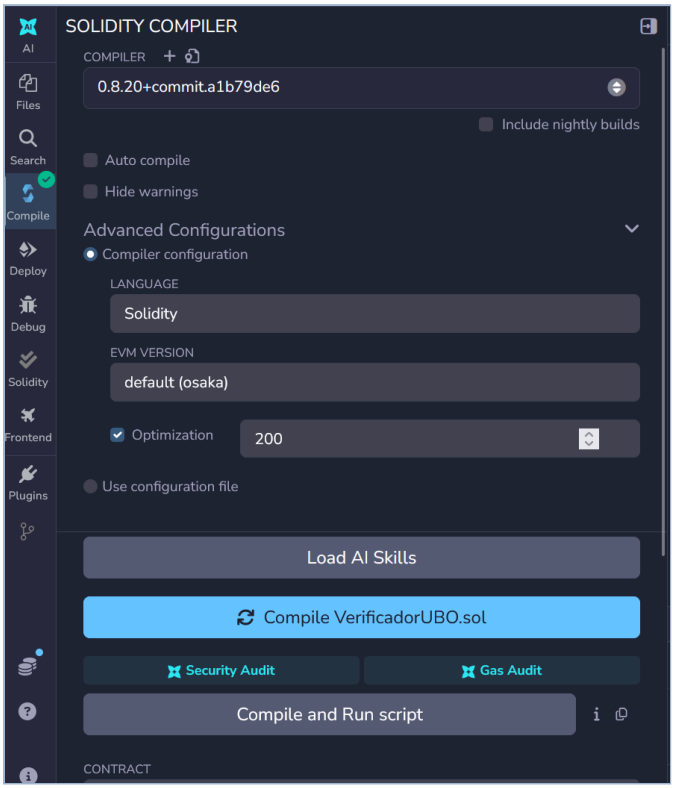
Sello Libertador: informe del Hito 1, Módulo C

Bloque 3: Verificación y enlace. Materia Criptografía y Blockchain 2026. Capturas y resultados de Remix IDE

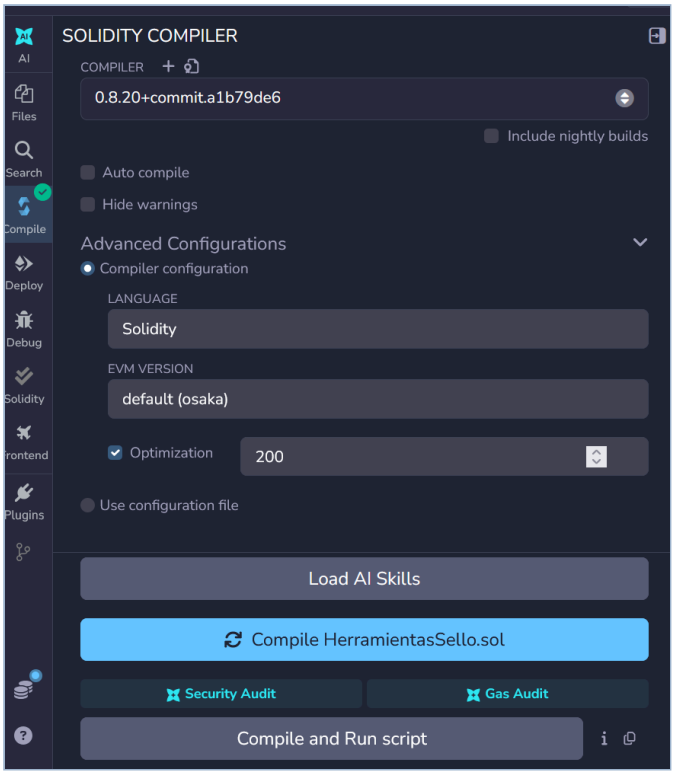
Equipo: Noé GUENGANT, Malo KERAUTRET, Lukas LAUDRAIN, Alex FOUGEROUX

1. Compilación

Compilador 0.8.20, EVM Version por defecto y optimización activada en 200. El icono Compile muestra la marca verde y no hay errores.

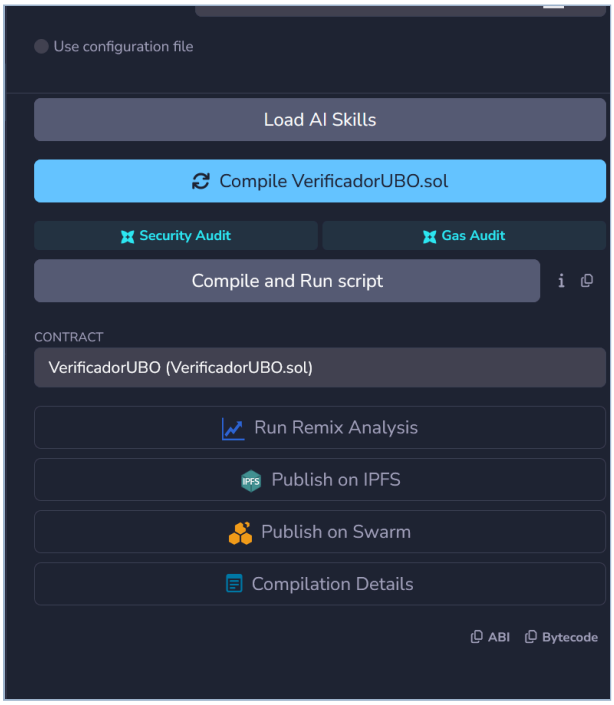


VerificadorUBO.sol



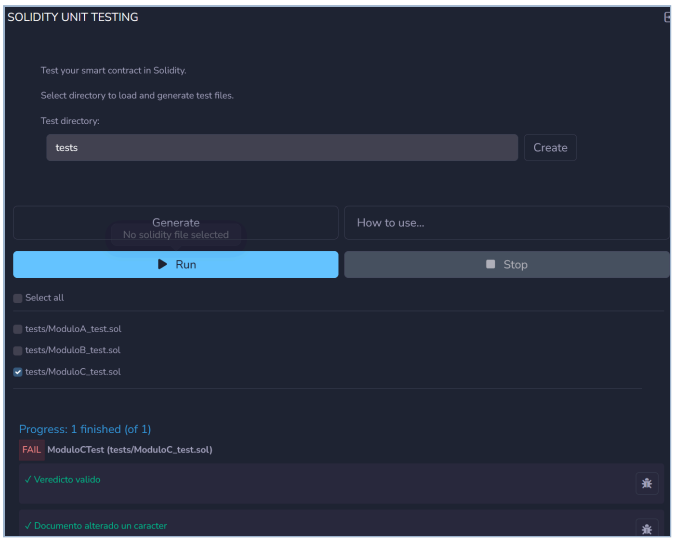
HerramientasSello.sol

Parte inferior del panel Compile para VerificadorUBO: sin errores ni advertencias, y el contrato aparece compilado.



2. Pruebas automáticas (Solidity Unit Testing)

Resultado de tests/ModuloC_test.sol: 2 pruebas en verde (Veredicto valido y Documento alterado un caracter) y 6 en rojo con el mensaje TODO, que son las de las semanas 2 y 3.





3. Guion manual

Tabla A: VerificadorUBO (sección 5.7 del enunciado)

Nota: `hashING = 0x0b8a7252dab48454160601ab2021f28afb4db1f0c2801c51d71eaba104107a62`, L = `0x55305a5341385150464e` (U0ZSA8QPFN).

Veredictos: 0 Valido, 1 Revocado, 2 Suspendido, 3 Inexistente, 4 EmisorNoAutorizado, 5 DocumentoAlterado.

#	Paso	Resultado esperado	Resultado obtenido
1	Desplegar `SimRegistroEmisores` y `SimRegistroTitulos`	Se despliegan sin error	Se desplegaron ✓ (bloques 20 y 21, estado 1)
2	Desplegar `VerificadorUBO(direccionSimTitulos, direccionSimEmisores)` (primero títulos, después emisores)	Se despliega sin error; `titulos()` y `registro()` devuelven esas direcciones	Se desplegó ✓ (bloque 22). Argumentos decodificados: titulos_ = SimRegistroTitulos (0xe289…157A), registro_ = SimRegistroEmisores (0x5e17…4Eff5)
3	En SimEmisores, desde cuenta 1: `autorizarEmisor(cuenta1, hashING)`	Marca verde; `estaAutorizado(cuenta1, hashING)` = `true`	Marca verde ✓. Evento EmisorAutorizado(cuenta1, hashING, 1791856541). `estaAutorizado(cuenta1, hashING)` devolvió `true`
4	Desde cuenta 1, en SimTitulos: `emitir(L, hashING, hashPrograma, 0x…01)`	Marca verde; `obtenerTitulo(L)` muestra estado 1 y compromiso `0x…01`	Marca verde ✓. Evento TituloEmitido(L, hashING, compromiso 0x…01, emisor = cuenta1)
5	`verificar(L, 0x…01)`	`0` (Valido)	Devolvió `0` ✓ (Valido)
6	`verificar(L, 0x…02)`	5 (DocumentoAlterado)	Devolvió 5 ✓ (DocumentoAlterado)
7	`verificar(otroLocalizadorNuncaEmitido, 0x…01)`	3 (Inexistente), sin revertir	Devolvió 3 ✓ (Inexistente), sin revertir
8	En SimTitulos: `suspender(L, 5)`; luego `verificar(L, 0x…01)`	2 (Suspendido)	`suspender` marca verde ✓, evento EstadoCambiado(L, 2, 5). `verificar` devolvió 2 ✓
9	En SimTitulos: `revocar(L, 2)`; luego `verificar(L, 0x…02)` (compromiso incorrecto)	1 (Revocado): el estado se comprueba antes que el compromiso	`revocar` marca verde ✓, evento EstadoCambiado(L, 3, 2). `verificar(L, 0x…02)` devolvió 1 ✓ (Revocado, aun con compromiso incorrecto)
10	Emitir otro título L2 como en el paso 4; en SimEmisores `fijarPeriodo(cuenta1, hashING, desde, 0)` con desde mayor que la fecha de emisión (p. ej. `block.timestamp + 1000000`); `verificar(L2, 0x…01)`	4 (EmisorNoAutorizado)	`emitir(L2)` marca verde ✓ (TituloEmitido). `fijarPeriodo(cuenta1, hashING, 99999999999, 0)` marca verde ✓. `verificar(L2, 0x…01)` devolvió 4 ✓ (EmisorNoAutorizado)

Extra (explicación, ya implementada): `verificarConExplicacion(L2, 0x…01)` en el paso 5 devuelve `(0, "Titulo valido emitido por la UBO")`.

Tabla B: HerramientasSello (valores de la sección 5.7)

Llamada	Resultado esperado	Por qué	Resultado obtenido
`byteAceptado(0)`	`true`	0 < 252	`true` ✓
`byteAceptado(251)`	`true`	último aceptado	`true` ✓
`byteAceptado(252)`	`false`	252 = 7 × 36 es el primer descartado	`false` ✓
`byteAceptado(255)`	`false`	255 ≥ 252	`false` ✓
`simboloDe(0)`	`0x30` («0»)	0 % 36 = 0	`0x30` ✓
`simboloDe(9)`	`0x39` («9»)	9 % 36 = 9	`0x39` ✓
`simboloDe(10)`	`0x41` («A»)	10 % 36 = 10	`0x41` ✓
`simboloDe(35)`	`0x5a` («Z»)	35 % 36 = 35	`0x5a` ✓
`simboloDe(36)`	`0x30` («0»)	36 % 36 = 0	`0x30` ✓
`simboloDe(252)`	revierte `ByteNoAceptado(252)`	byte no aceptado	Revirtió con `ByteNoAceptado(252)` ✓
`derivarLocalizador(0x00…01)`	`0x584539414149374e324d` («XE9AAI7N2M»)	determinista	`0x584539414149374e324d` ✓
`derivarLocalizador(0x00…01)` otra vez	igual al anterior	misma semilla, mismo resultado	`0x584539414149374e324d` ✓ (igual)
`derivarLocalizador(0x00…02)`	`0x53465259494f5a385133` («SFRYIOZ8Q3»)	semilla distinta, localizador distinto	`0x53465259494f5a385133` ✓

Localizadores derivados (calculados con una implementación independiente de keccak256 comprobada contra `keccak256("ING")` del enunciado): **XE9AAI7N2M** (semilla `0x00…01`) y **SFRYIOZ8Q3** (semilla `0x00…02`).

Para pasar la semilla en Remix: `0x0000000000000000000000000000000000000000000000000000000000000001`.